

LLMOrchestrator

LLMOrchestrator

Un plan de contrôle unique pour chaque agent de codage CLI sans interface.

Résumé

LLMOrchestrator est un module Go autonome et réutilisable conçu pour lancer, gérer et communiquer avec des agents CLI sans interface (OpenCode, Claude Code, Gemini, Junie, Qwen Code) via un protocole hybride combinant tubes et fichiers. Il intègre des disjoncteurs par agent, une sélection multi-fournisseurs pluggable, une abstraction i18n découplée et des garanties anti-biais de test.

Description courte

Un module Go réutilisable offrant une interface unifiée pour lancer et piloter plusieurs agents CLI alimentés par LLM via un protocole hybride tubes+fichiers. Mise en commun d'agents sécurisée pour les threads, avec disjoncteurs et stratégies de routage sélectionnables, délibérément agnostique vis-à-vis des consommateurs, et doté d'un traducteur i18n pluggable.

Description longue

LLMOrchestrator constitue une infrastructure partagée pour orchestrer des agents de codage CLI sans interface — cette couche technique dont tout système multi-agents a discrètement besoin et que l'on réinvente généralement mal. Plutôt que chaque projet réimplémente le lancement de processus, le cadrage des messages et l'analyse des résultats pour des outils comme OpenCode, Claude Code, Gemini CLI, Junie ou Qwen Code, ce module propose une interface Agent unifiée, un AgentPool sécurisé pour les threads et un MultiProviderPool qui fédère des agents issus de plusieurs fournisseurs derrière une façade unique. Le

routage est pluggable via un AgentSelector — tourniquet évitant les fournisseurs ne répondant pas aux exigences, ou ordre de préférence avec repli — de sorte que la répartition des tâches relève d’une politique choisie, et non d’une hypothèse figée. Chaque agent concret est un adaptateur léger reposant sur un BaseAdapter partagé, qui gère l’intégralité du cycle de vie du processus : démarrage avec configuration des tubes, arrêt gracieux via SIGTERM puis SIGKILL, redémarrage et surveillance de l’état — autant d’aspects fastidieux et sources d’erreurs, résolus une fois pour toutes.

La communication est délibérément hybride, adaptant le transport à la tâche. Un transport par tube achemine des messages JSON délimités par des sauts de ligne, avec un délai de lecture par requête et une limite de taille de réponse pour les échanges interactifs rapides, tandis qu’un transport par fichiers utilise des répertoires partagés boîte de réception/boîte d’envoi par session pour les artefacts volumineux ou persistants, qui n’ont pas leur place dans un tube. La résilience n’est pas un ajout après coup — elle est structurelle : un disjoncteur par agent s’ouvre après trois échecs consécutifs pour une période de refroidissement de 60 secondes avant un test en demi-ouverture, et un moniteur de santé en arrière-plan sonde les agents afin qu’un agent hors service puisse se rétablir sans attendre qu’un trafic entrant ne le détecte. L’acquisition d’un agent dans le pool se bloque sur une variable de condition plutôt que de consommer inutilement du CPU en attente active, et l’analyseur de réponses est sans état et sécurisé pour les appels concurrents. Le module est strictement découplé — aucune spécificité liée aux consommateurs ne peut s’y infiltrer — et chaque chaîne visible par l’utilisateur passe par un Translator pluggable, avec un NoopTranslator qui retourne les identifiants de message tels quels afin qu’une traduction manquante soit immédiatement visible plutôt que masquée.

Pourquoi nous l’avons conçu

Tout système multi-agents doit pouvoir lancer et communiquer de manière fiable avec des agents CLI. Réinventer à chaque projet le lancement, le cadrage, l’analyse et la gestion des échecs est à la fois coûteux et source d’erreurs. LLMOrchestrator centralise ces fonctionnalités dans un module découplé et réutilisable, dont la responsabilité spécialisée en fait un composant réutilisable — et cette réutilisabilité serait compromise dès lors que des spécificités liées aux consommateurs s’y immisceraient.

Contenu

Pourquoi c'est un changement de paradigme

Passer de « gérer une armée d'agents CLI hétérogènes » d'un bricolage d'ingénierie sur mesure par projet à une simple importation de bibliothèque — avec la mutualisation, le *circuit breaking*, la gestion du cycle de vie et un routage pluggable déjà résolus et éprouvés. Et comme ses tests anti-bidon évaluent le système réel de bout en bout plutôt que de se contenter d'un « ça compile », vous obtenez une abstraction sur laquelle vous pouvez vraiment compter en situation de concurrence et de défaillance, et non une solution qui se contente d'avoir l'air correcte sur un schéma.

Ce qui est innovant

- **Protocole hybride canal+fichier** — vitesse interactive (lignes JSON via stdin/stdout, délais de lecture, limites de réponse) *et* échange durable basé sur des fichiers (boîte de réception/boîte d'envoi/partagé) pour les gros artefacts, afin de ne jamais avoir à choisir entre latence et durabilité.
- **Pool multi-fournisseurs avec sélecteurs pluggables** — une façade unique pour plusieurs fournisseurs CLI, avec un routage en tourniquet ou par ordre de préférence défini par politique plutôt que figé.
- **Disjoncteur par agent + moniteur de santé en arrière-plan** — dégradation *et* récupération automatiques (3 échecs → ouverture pendant 60 s → sonde en demi-ouverture), de sorte qu'un agent instable est isolé puis réintégré discrètement sans intervention manuelle.
- **Pooling sans attente active** — Acquire bloque sur sync.Cond jusqu'à ce qu'un agent sain et compatible se libère ou que le contexte soit annulé, évitant ainsi toute consommation inutile de CPU.
- **Découplage strict + i18n anti-bidon** — le NoopTranslator retourne les identifiants de message tels quels, rendant impossible de passer à côté d'une traduction manquante au lieu de laisser un blanc silencieux.
- **Sécurité par défaut** — une liste blanche de chemins binaires élimine l'interpolation shell et donc toute surface d'injection de commande, renforcée par une protection contre le *path traversal*, une limite de 1 Mio pour les réponses afin d'éviter les sorties incontrôlées, et le masquage des clés API dans les logs.
- **Harnais de test anti-bidon** — des allers-retours réels disque/JSON/parseur sur cinq locales, avec une porte de mutation couplée qui doit retourner un code de sortie non nul si la

fonctionnalité est défectueuse — un test qui prouve qu’il peut effectivement échouer.

Principaux défis techniques et solutions apportées

- **E/S fiables des processus agents.** Communiquer avec un processus CLI lancé est trompeusement complexe ; solution apportée par un transport hybride canal+fichier, un contrat de message/parseur défini pour que les deux côtés s’accordent sur le format de transmission, et un BaseAdapter centralisant tout le cycle de vie du processus, y compris un délai de grâce SIGTERM qui escalade vers un SIGKILL en dernier recours.
- **Concurrence sans attente active.** Résolu grâce à un AgentPool basé sur un mutex et une variable de condition, où Acquire reste en veille jusqu’à ce qu’un agent compatible se libère, couplé à un parseur sans état et sans effets de bord, sécurisé pour être appelé depuis plusieurs goroutines simultanément.
- **Isolation des défaillances des fournisseurs.** Résolu de sorte qu’un fournisseur défaillant ne puisse pas entraîner les autres : des disjoncteurs par agent limitent l’impact, et une goroutine de surveillance de santé pilote la récupération même en l’absence de requêtes pour la déclencher.
- **Preuve de la justesse, pas seulement de la compilation.** Résolu avec un exécuteur de *Challenge* : des dizaines d’invariants testés en en/sr/ja/es/de sur le système réel, plus une porte de mutation couplée (LLMORCH_MUTATE_RUNNER=1 doit échouer → sortie 99 du wrapper) qui casse délibérément la fonctionnalité pour prouver que la porte elle-même n’est pas un leurre.
- **Localisation sans échec silencieux.** Résolu grâce à la couture du NoopTranslator à identifiants verbatim et à l’injection de traducteurs par consommateur, de sorte qu’une lacune dans les traductions soit toujours visible plutôt que masquée.

Contenu

Pile technologique

- **Go (1.25)** — choisi pour sa gestion de premier ordre de la concurrence et son contrôle précis des processus, des atouts essentiels pour orchestrer des processus d’agents en temps réel ; il implémente le module, ses adaptateurs d’agents, les transports et le parseur.
- **Go stdlib uniquement (+ testify, yaml.v3)** — un choix délibéré visant à minimiser la surface des dépendances et à éviter toute intégration des SDK LLM, afin que le module reste

léger et intégrable dans n'importe quel consommateur sans alourdir ce dernier avec des dépendances externes.

- **Transport par tube (JSON-lines via stdio)** — retenu pour son efficacité dans les échanges interactifs rapides, renforcé par des délais de lecture et des limites de taille de réponse afin d'éviter qu'un agent bloqué ou incontrôlable ne paralyse l'appelant.
- **Transport par fichiers (boîtes d'envoi/réception/partagées)** — privilégié pour l'échange durable de gros artefacts par session, là où un tube serait inadapté.
- **sync.Mutex/sync.Cond** — utilisés pour implémenter une acquisition équitable et bloquante du pool d'agents, sans attente active.
- **Disjoncteur + HealthMonitor** — combinés pour offrir une résilience par agent *et* une récupération active, bien au-delà d'une simple détection des défaillances.
- **Traducteur pkg/i18n** — choisi comme interface de localisation découplée, permettant de maintenir les chaînes spécifiques aux consommateurs en dehors du cœur du module.
- **Harnais de tests (challenges/runner) + Makefile (test -race, fuzz, cover)** — retenus pour une vérification rigoureuse et étayée par des preuves, incluant la détection de conditions de course et le fuzzing du parseur, afin de démontrer la robustesse dans des conditions adverses plutôt que de la supposer.

Statut et notes de transparence

- **Statut : bêta.** Un module réutilisable et découplé, intégré en tant que sous-module dans plusieurs projets Helix/vasic.
Licence : Apache-2.0 ; le dépôt GitHub est public.
- Les métadonnées des modèles proviennent de LLMsVerifier, relayées via HelixQA ; ce module n'importe pas directement LLMsVerifier/VisionEngine/DocProcessor. Les piles technologiques mentionnées dans le CLAUDE.md de l'application parente (Gin/PostgreSQL/etc.) décrivent helix_code, et non ce module.

Niveau de priorité : Helix-primaire (grappe LLM-infrastructure — module réutilisable découplé). Classé après HelixTrack.